

CAMPUS RECRUITMENT TRAINING - PHASE 2

POST-ASSESSMENT

Name: Soham Nakhate
Roll No. : C-45
Section: C
Branch: CSE
GITHUB LINK

Q1. Predict the output. What will print to the console when this Spring Boot application starts? Write the output in order and give a one-line reason for each. [4M]

```
@Component
public class MyBean {
    public MyBean() {
        System.out.println("Constructor called");
    }

    @PostConstruct
    public void init() {
        System.out.println("Init called");
    }
}
```

Ans: OP:-

1. Constructor called
2. Init called

Reason: First the MyBean object is created as @Component annotation is called on it and then after the construction of the bean, the method with the @PostConstruct annotation (init())is called.

Q2. The endpoint below compiles and runs but always returns an empty response body. [3M]
Identify the mistake and write the corrected line only.

```
@Controller
@RequestMapping("/api/users")
public class UserController {
    @Autowired UserService service;

    @GetMapping("/{id}")
    public ResponseEntity<User> getUser(@PathVariable Long id) {
        User u = service.findById(id);
        return new ResponseEntity<>(HttpStatus.OK);
    }
}
```

Ans: the Response entity is created only with HttpStatus.OK and not along with user u hence it must be showing empty response body. It should be
return new ResponseEntity<>(u, HttpStatus.OK);

Q4. What annotation should be added to the transfer() method below, and why? Write the annotation and explain in 2 lines what happens if it is missing.

[3M]

```
@Service
public class BankService {
    @Autowired AccountRepository repo;

    public void transfer(Long fromId, Long toId, double amount) {
        Account from = repo.findById(fromId).orElseThrow();
        Account to = repo.findById(toId).orElseThrow();
        from.setBalance(from.getBalance() - amount);
        to.setBalance(to.getBalance() + amount);
        repo.save(from);
        repo.save(to);
    }
}
```

Ans:

@Transactional is to be added.

Without @Transactional, each save() may execute separately, so if one update succeeds and the other fails, the database can become inconsistent.

@Transactional ensures both account updates happen as a single transaction (all succeed or all rollback).

Q5 CODING

Q6. Look at the JWT filter below. (a) Explain in 2 lines what this filter does. (b) There is one bug — the token extracted from the header will always be invalid. Find it and write the corrected line.

[5M]

```
String header = req.getHeader("Authorization");
if (header == null || !header.startsWith("Bearer ")) {
    chain.doFilter(req, res);
    return;
}
String token = header.substring(6); // extract token
String username = jwtUtil.extractUsername(token);
if (username != null) {
    UsernamePasswordAuthenticationToken auth =
        new UsernamePasswordAuthenticationToken(username, null, null);
    SecurityContextHolder.getContext().setAuthentication(auth);
}
chain.doFilter(req, res);
```

ANS:

This filter checks the Authorization header for JWT Token that has Bearer in the starting. If the token is valid, it extracts the username by extractUsername and sets the authentication in the security context.

String token = header.substring(7); -> because it is given that header should start with ("Bearer ") i.e. 7 Characters.

Q7. Write a Spring Data JPA repository method using @Query to find all Product entities where price is less than a given value and category matches a given string. Use named parameters.

ANS:

```
@Query("SELECT p FROM Product p WHERE p.price < :price AND p.category = :category")
```

```
List<Product> findByPriceLessThanValueAndCategory(@Param("price") double price, @Param("category") String category);
```

Q8. Explain the N+1 query problem in JPA in your own words with a simple example. What is the most common cause? Write one line of code that fixes it in a @Query method using JOIN FETCH.

ANS:

In N+1 problem, the JPA first executes 1 query and then goes to parent entities and then executes other N queries.

Eg. an Order entity has a list of Item entities.

```
List<Order> orders = orderRepo.findAll();
```

First query fetches all orders. Then, for each order, JPA separately fetches its items.

If there are 10 orders, total queries = 1 + 10 = 11 queries.

Most Common Cause

The most common cause is lazy-loaded relationships (FetchType.LAZY) accessed inside a loop.

Fix using JOIN FETCH

```
@Query("SELECT o FROM Order o JOIN FETCH o.items")
```

Q9. Write a SQL query to find the second highest salary from a table employees(id, name, salary). Use a subquery approach. Then write the same logic as a JPQL @Query method in a JPA repository.

ANS:

SQL

```
SELECT MAX(salary) FROM employees
```

```
WHERE salary < (
SELECT MAX(salary) FROM employees
);
JPQL
@Query("SELECT MAX(emp.salary) FROM Employee emp WHERE emp.salary < " +
"(SELECT MAX(emp.salary) FROM Employee emp)" )
double find2ndHighestSal();
```

Q10. Given a Student entity with fields name, email, and city, write FOUR Spring Data JPA derived query method names (no @Query) for the following: (a) find by city, (b) find by name and city, (c) find by email containing a keyword, (d) count students in a city. Write the method signatures only

ANS:

(a) Find by city

```
List<Student> findByCity(String city);
```

(b) Find by name and city

```
List<Student> findByNameAndCity(String name, String city);
```

(c) Find by email containing a keyword

```
List<Student> findByEmailContaining(String keyword);
```

(d) Count students in a city

```
long countByCity(String city);
```

Q11. A developer runs the following code to load 10 orders and print their items. The console shows 11 SQL queries instead of 1. What is this problem called? Write the line of code that causes it and the one-line fix.

```
List<Order> orders = orderRepo.findAll();
for (Order o : orders) {
    System.out.println(o.getItems().size());
}
```

ANS:

This is called the N+1 Problem. In this, JPA executes the query 1 and then fetches the Entity, then the N queries are additionally executed hence the count of queries is added by 1.

Fix:

```
@Query("SELECT o FROM Order o JOIN FETCH o.items")
```

Q12. Write the application.yml configuration and the main class annotation needed to register a Spring Boot service named product-service with a Eureka server at <http://localhost:8761/eureka>.

application.properties

```
spring:
  application:
    name: product-service
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
```

Main Class

```
@SpringBootApplication
@EnableEurekaClient
public class ProductServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(ProductServiceApplication.class, args);
    }
}
```

Q13. What is a Circuit Breaker in microservices? Why is it needed? Write a Spring Boot method that calls an external payment-service using RestTemplate and applies a Resilience4j @CircuitBreaker with a fallback method that returns a default PaymentResponse object.

ANS:

A Circuit Breaker is a design pattern used in creation of microservices that stops the repeated calls to the failed service and blocks the requests if the failure exceeds a threshold value.

It helps in designing stable systems and allow fallback responses when a service is down.

CODE:

```
@Service
public class PaymentServiceClient {

    @Autowired
```

```

private RestTemplate restTemplate;

@CircuitBreaker(name = "paymentService", fallbackMethod =
"fallbackPayment")
public PaymentResponse callPaymentService() {

    String url = "http://payment-service/api/pay";

    return restTemplate.getForObject(url, PaymentResponse.class);
}

// Fallback method
public PaymentResponse fallbackPayment(Exception ex) {

    PaymentResponse response = new PaymentResponse();
    response.setStatus("FAILED");
    response.setMessage("Payment service is unavailable");

    return response;
}
}

```

Q14. What is an API Gateway and what are 2 benefits of using one? Write a Spring Cloud Gateway application.yml that routes requests starting with /api/orders/** to order-service and /api/users/** to user-service. Use lb:// for load-balanced URIs.

ANS :

An API Gateway is a design pattern that acts as a single entry point for all the client requests to the microservices. It helps in load balancing and in centralized routing and security handling.

CODE in : app_q14.properties

Q15. What is Service Discovery in microservices and why is it needed instead of hardcoding IP addresses? Name the tool you used in class. Write a RestTemplate bean configuration that enables it to resolve service names automatically

ANS:

Service Discovery is a mechanism through which the microservices automatically register themselves dynamically. Hardcoding IP addresses is unreliable because they are prone to change during restarts and deployments. Service discovery provides dynamic lookup and load balancing.

For Service Discovery, we used Netflix Eureka Server. In this we added different microservices and checked if their status was UP or DOWN.

```
@Configuration
public class AppConfig {
    @Bean
    @LoadBalanced
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}
```

Q16. You have a monolithic Spring Boot app with modules: UserModule, OrderModule, ProductModule. Your team wants to split it into microservices. (a) List 3 benefits of this split. (b) List 2 challenges. (c) Briefly explain how Eureka and an API Gateway work together so a React frontend can talk to all three services through a single URL.

ANS:

A. BENEFITS:

1. Independent Deployment
2. Better Scalability
3. Technology flexibility

B. CHALLENGES

1. Increased Complexity
2. Distributed system issues

C.

Each microservice (user-service, order-service, product-service) registers itself with Netflix Eureka. The API Gateway queries Eureka to discover service locations dynamically. The React frontend sends all requests to a single Gateway URL, and the Gateway routes requests to the correct microservice internally.

Q17. Write a JUnit 5 + Mockito unit test for the `getById()` method below. Your test should:
(a) use `@Mock` and `@InjectMocks`, (b) stub the repository to return a sample `Product`, and
(c) assert that the returned product name equals "Laptop".

[5M]

```
@Service
public class ProductService {
    @Autowired ProductRepository repo;

    public Product getById(Long id) {
        return repo.findById(id).orElseThrow();
    }
}
```

ANS:

CODE IN REPO.

Q18. What is the difference between `@Mock` (Mockito) and `@MockBean` (Spring Boot Test)? When would you use each? Write one example of a test class declaration that uses `@MockBean` correctly with `@SpringBootTest`.

ANS:

`@Mock` (Mockito)- Creates a mock object using Mockito only.Used in unit tests without loading the Spring context.

`@MockBean` (Spring Boot Test)- Creates and injects a mock into the Spring Application Context.Used in integration tests with `@SpringBootTest` or web-layer tests.

We Use `@Mock` for fast isolated unit testing of a single class.Use `@MockBean` when testing Spring-managed beans and replacing real dependencies in the context.

```
@SpringBootTest
class ProductServiceIntegrationTest {
    @Autowired
    private ProductService service;
    @MockBean
    private ProductRepository repo;
}
```

Q19. Write a Dockerfile for a Spring Boot application. The JAR file is named `app.jar` and is inside the `target/` folder. Use `eclipse-temurin:21-jre` as the base image. The app runs on port 8080. Then write the docker build and docker run commands to build the image as `myapp:latest` and run it.

ANS:

Dockerfile:


```
FROM eclipse-temurin:21-jre

WORKDIR /app

COPY target/app.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]

Docker run commands:

docker build -t myapp:latest .
docker run -p 8080:8080 myapp:latest
```

Q20. Write a docker-compose.yml with two services: (a) db – using the postgres:15 image with environment variables for DB name, user, and password, and (b) app – using the myapp:latest image, mapping port 8080, connecting to the db service, and starting only after db starts using depends_on.

ANS:

```
services:
  db:
    image: postgres:15
    environment:
      POSTGRES_DB: mydb
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: password

  app:
    image: myapp:latest
    ports:
      - "8080:8080"
    depends_on:
      - db
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/mydb
      SPRING_DATASOURCE_USERNAME: postgres
      SPRING_DATASOURCE_PASSWORD: password
```

Q21. Write a JavaScript async function loadUsers() that fetches data from GET /api/users using the fetch API, checks if the response is successful (response.ok), and either renders a list of user names into aelement or shows an error message in a element.

ANS:

```

async function loadUsers() {
  const userList = document.getElementById("user-list");
  const error = document.getElementById("error");
  try {
    const response = await fetch("/api/users");
    if (!response.ok)
      throw new Error("Failed to fetch users");

    const users = await response.json();
    userList.innerHTML = "";
    users.forEach(user => {
      const li = document.createElement("li");
      li.textContent = user.name;
      userList.appendChild(li);
    });
  } catch (err) {
    error.textContent = err.message;
  }
}

```

Q22. The React component below has a bug that causes it to fetch data in an infinite loop. [5M]
 (a) Identify exactly what is wrong. (b) Fix it. (c) Add a simple loading state that shows 'Loading...' while data is being fetched.

```

function UserList() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    fetch('/api/users')
      .then(res => res.json())
      .then(data => setUsers(data));
  }); // <-- notice anything?

  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}

```

ANS:

The `useEffect()` function is not dependent on any array as `[]` is passed in it. And the `setUsers(data)` re-renders with everytime hence it goes in Infinite Loop.

CODE : (Fixed and adding loading state)

```

function UserList() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch('/api/users')
      .then(res => res.json())
      .then(data => {
        setUsers(data);
        setLoading(false);
      });
  }, []);

  return <div>
    {loading ? <div>Loading...</div> : <ul>
      {users.map(u => <li key={u.id}>{u.name}</li>)}
    </ul>}
  </div>;
}

```

```
        setLoading(false);
    });
}, []);

if (loading) {
    return <p>Loading...</p>;
}

return (
    <ul>
        {users.map(u => (
            <li key={u.id}>{u.name}</li>
        ))}
    </ul>
);
}
```